

Functional vs Non-Functional Requirements

1. FUNCTIONAL REQUIREMENTS (What the system must do)

Key Points:

- If it describes a *task, process, or user action*
- Define the **main functions** or **services** the system provides.
- Specify how the system should **react to inputs** and **produce outputs**.
- Directly linked to **user needs** and **business processes**.
- Visible to end users

Examples:

- The system must allow users to **log in** using a valid ID and password.
- Customers can **place, edit, or cancel orders**.
- The system generates **monthly financial reports**.
- Admin can **update menu items and prices**.
- The user can **search** for products by name or category.

2. NON-FUNCTIONAL REQUIREMENTS (How the system should perform)

Key Points:

- Its *quality attributes* and *constraints(limitations)*.
- Describe **system qualities** like performance, security, and usability.
- Define the **standards** that make the system efficient, secure, and reliable
- Support the **user experience** and **system stability** rather than new features.
- Usually **measurable** (e.g., "Response time < 2 seconds").
- Often behind-the-scenes

Examples:

- The system should load within **3 seconds**.
- The application must support **1,000 concurrent users**.
- Data must be **encrypted** during transmission.
- The interface should be **mobile-friendly and accessible**.
- System uptime should be **99.9%**.

TYPES OF NON-FUNCTIONAL REQUIREMENTS

1. Performance:

Focus: *Speed and responsiveness* of the system.

Key points:

- Measures how fast the system responds to user actions.
- Includes response time, throughput, and processing speed.
- Often expressed in measurable terms (e.g., "load in < 3 seconds").

Example: *System must process 100 transactions per second*

2. Reliability

Focus: *Consistency and dependability* of system operations.

Key points:

- Ensures the system performs correctly over time.
- Includes **error recovery**, **fault tolerance**, and **uptime**.

Example: *The system must maintain 99.9% uptime.*

3. Portability

Focus: *System adaptability* to different environments.

Key points:

- Determines how easily the system can run on multiple platforms.
- Includes cross-device and cross-OS compatibility.

Example: *Application runs on Windows, macOS, and Android.*

4. Usability

Focus: *Ease of use and user experience.*

Key points:

- Measures how intuitive and accessible the system is.
- Includes interface design, help features, and accessibility.

Example: *Users should complete an order in 3 clicks or less.*

5. Security

Focus: *Protection of data and access control.*

Key points:

- Ensures data privacy, authentication, and authorization.
- Guards against hacking and unauthorized use

Example: *Passwords must be encrypted before storage.*

6. Maintainability

Focus: *Ease of updating and fixing the system.*

Key points:

- Well-documented code and modular structure help maintenance.
- Reduces downtime during updates or bug fixes

Example: *Developers can add features without rewriting the system.*

7. Scalability

Focus: *Growth and performance under increasing load.*

Key points:

- The system should handle more users, data, or transactions smoothly.
- Includes both vertical and horizontal scalability

Example: *System supports 10,000 concurrent users.*

8. Availability

Focus: *System uptime and readiness.*

Key points:

- Ensures the system is operational when needed.
- Tied to backup, failover, and recovery mechanisms

Example: *System available 24/7 except for maintenance windows.*

9. Compatibility

Focus: *Ability to work with other systems or tools*

Key points:

- System can share data or connect with external software.
-
- Important for APIs, databases, and device integration

Example: *System integrates with PayPal and Google Maps APIs.*

10. Efficiency

Focus: *Optimal use of system resources.*

Key points:

- Efficiently manages CPU, memory, and bandwidth.
-
- Prevents system lag or crashes during peak use

Example: *App uses less than 100MB memory during operation*